

RoleTect

Zero Setup, Lightweight L^AT_EX IDE & Privacy-Focused LLM
Orchestrated Career Vault

Technical Architecture & Product Case Study

Author:

Md. Ramjan Miah
(RoleTect Source Code)

Date:

June 2026

1 Executive Summary

In today’s hyper-competitive job market, candidates face a critical paradox: securing interviews requires dynamically tailoring resumes and cover letters to defeat Applicant Tracking Systems (ATS), yet the manual workflow to achieve this is fragmented, exhausting, and insecure. This paper introduces the **LLM-Orchestrated, Privacy-First, Centralized Job Application Vault**, a revolutionary paradigm that unifies sovereign user data control with automated document optimization.

Compounding the workflow friction, modern **researchers and professionals** are bottlenecked by heavy typesetting environments, traditionally requiring bloated **5GB+ local LaTeX distributions** or yielding data custody to third-party cloud compilers. We resolve this dilemma by using a novel **Local-First, Lightweight LaTeX Compilation Engine**. By eliminating heavy dependency overhead, this system slashes local installation footprints from gigabytes to a self-contained, zero-configuration runtime, ensuring local document compilation occurs entirely client-side with **zero-data leakage**.

The Problem: Workflow Fragmentation & Security Risks

- **Operational Fatigue:** Candidates and researchers must navigate a disjointed pipeline of manual text extraction, cloud LLM portals, external $\text{\LaTeX}$ compilers, and third-party job trackers. This friction causes severe **workflow bottlenecks**.
- **The $\text{\LaTeX}$ Bloat Dilemma:** Legitimate document typesetting traditionally forces users to install massive, **5GB+ local $\text{\LaTeX}$ distributions**. This heavy overhead discourages adoption and creates immense friction for rapid, iterative document generation.
- **Data Vulnerability:** Centralizing highly granular **Personally Identifiable Information (PII)** on third-party SaaS platforms exposes users to corporate data monetization, systemic brokering, and **targeted phishing campaigns**.

1.1 The RoleTect Solution

RoleTect introduces a definitive shift away from cloud-dependent and resource-heavy utilities. Architected as a cross-platform, **local-first** ecosystem (interactive web extension + Tauri v2 core), it completely centralizes the application lifecycle while ensuring absolute data sovereignty.

We have effectively solved the $\text{\LaTeX}$ overhead problem by replacing traditional, bloated engines with **Tectonic**, a **streamlined, self-contained compilation runtime**. This allows for professional-grade document rendering without the multi-gigabyte footprint. The entire **operational lifecycle runs entirely on-device** to eliminate data privacy risks. Your data never leaves your machine, and your **API keys** are **cryptographically** secured using OS-level **AES encryption**.

2 The Deep-Dive Problem Statement

The modern job application pipeline is fundamentally fractured. To clear technical screeners, candidates must continuously tailor professional documentation to the semantic footprint of each vacancy—introducing an exhausting, multi-platform loop that compromises privacy, organization, and efficiency.

2.1 The Multi-Platform Friction Loop

Standard application workflows require manual execution across four disjointed tiers:

1. **Data Ingestion:** Copying unstructured job data from web portals while battling localized copy-blocks and dynamic UI barriers.
2. **Context Orchestration:** Pasting payloads into web LLMs and designing custom prompts to execute profile tailoring.
3. **Compilation & Rendering:** Migrating output code to cloud $\text{\LaTeX}$ clusters (e.g., Overleaf) and waiting on server queues to debug typesetting.
4. **State Tracking:** Downloading the PDF while manually logging metrics (salaries, dates) into a separate dashboard like Teal.

This environment-hopping breaks data continuity. Once a static PDF is downloaded, the link between its compilation state, markup variations, and the source job description is lost, making it difficult to match the precise resume variation during a callback.

2.2 The Privacy Paradox and PII Exposure

To bypass these bottlenecks, candidates frequently surrender data sovereignty to centralized SaaS platforms. Hosting comprehensive professional, financial, and educational records on remote cloud clusters triggers critical vulnerabilities:

- **Data Monetization:** Platforms routinely subsidize operations by aggregating, processing, and brokering granular candidate profiles to third-party marketing frameworks and recruitment syndicates.
- **Security Exploitation:** Secondary distribution of Personally Identifiable Information (PII) directly fuels targeted spam, coordinated phishing, fraudulent employment scams, and credential exposure.

2.3 Technical Barriers & Ecosystem Paywalls

Existing open-source or self-hosted alternatives present equally steep boundaries. Securing data privacy through **private deployment requires heavy systems engineering: local Docker networks, hosting dense runtimes, or managing local TeX Live environments that exceed 3-5 GB** in disk space. This forces **non-technical applicants to choose between convenience and privacy**. Furthermore, commercial SaaS providers exploit this gap by **locking high-fidelity visual assets (e.g., Mermaid.js architecture flowcharts) behind premium subscription paywalls or throttling export resolutions**.

2.4 Ecosystem Performance Metric Analysis

RoleTect decouples professional asset production from traditional SaaS platform subscriptions. Table 1 provides an operational breakdown of RoleTect’s technical advantages:

Table 1: System Performance & Structural Comparison

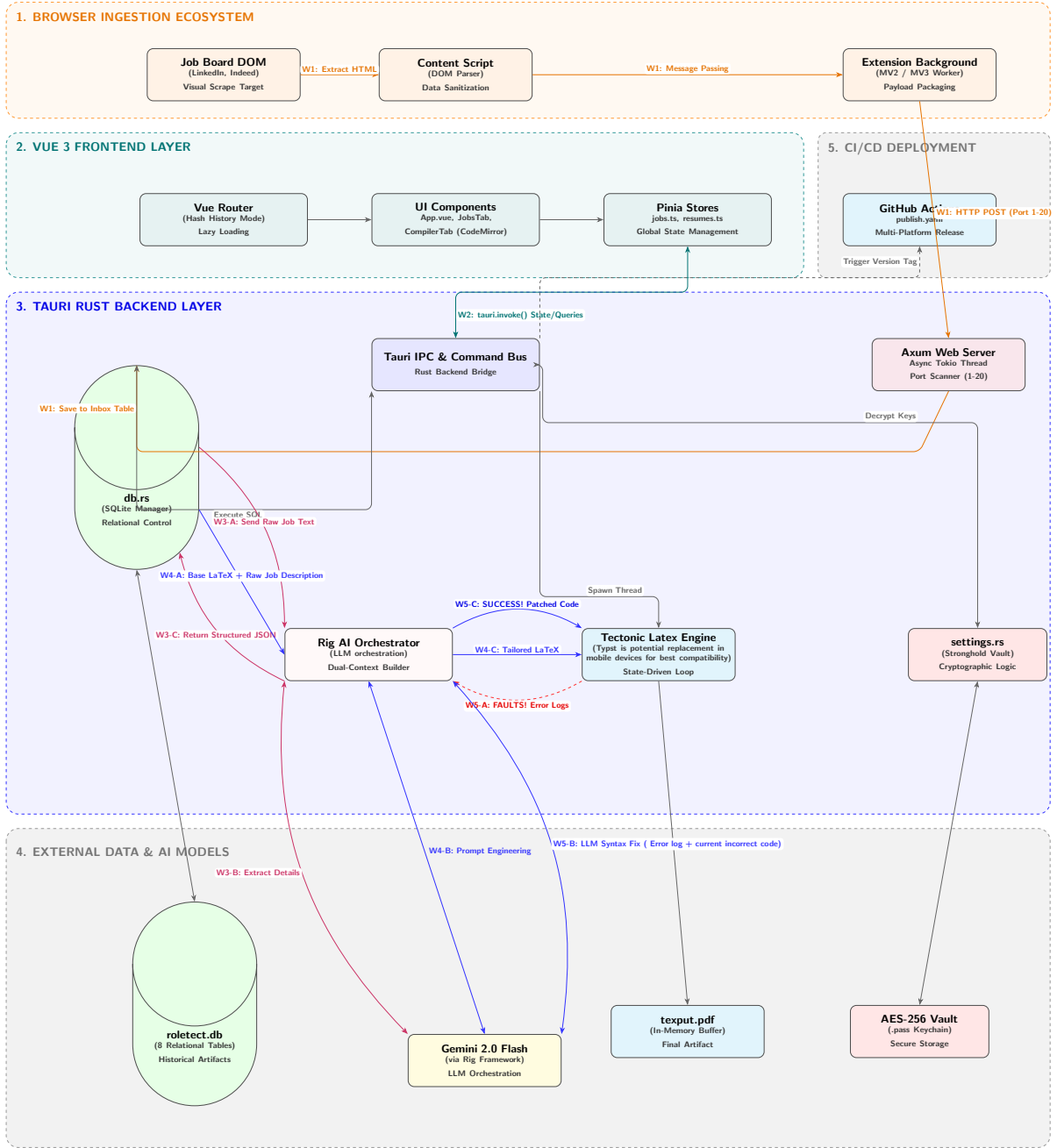
Evaluation Metric	Centralized SaaS (Teal/Overleaf)	RoleTect Ecosystem
Monthly Overhead Cost	\$10 – \$30 / month recurring	\$0.00 (Pay-per-token or Free)
Privacy Vulnerability	High Risk (PII Cloud Storage)	Zero Risk (OS Keyring / AES-256)
Model Independence	Proprietary / Locked-In Vendor	Total Freedom (AWS Bedrock/Ollama local)
Compilation Latency	1 – 10 Seconds (Network Dependent)	Very low (On-Device Cache)
Asset Resolution Limit	Restricted / Paywalled	Uncapped (Up to 16K Vector Exports)

3 System Architecture & Workflow Blueprint

The structural interactions between RoleTect’s local micro-server data ingress, Vue 3 Pinia state stores, **dynamic model orchestration**, and **self-healing LaTeX compilation loops** are mapped systematically below. This diagram explicitly visualizes the **5 primary AI and systemic workflows** that drive the application’s full potential.

3.1 Architectural Component Specifications

The vector topology detailed above demonstrates a robust decoupling of structural concerns designed for highly competitive technical validation, explicitly tracking all cross-stack workflows:



4 Technical Architecture & Ingestion Subsystems

RoleText is a local-first, desktop-centric suite built on Tauri v2. To eliminate the fragility of standard input polling, the desktop core transforms the host machine into an isolated micro-web ingress gateway using a custom asynchronous network architecture.

4.1 The Local Asynchronous Micro-Server Pipeline

During bootstrapping, the core Tauri binary spins up a dedicated multi-threaded worker space powered by the `tokio::runtime` async environment, initializing a high-performance, lightweight Axum web router. This local node functions as a non-blocking receiver for incoming job descriptions captured from the browser.

Rather than routing data through cloud middleware or unencrypted temp-files, the internal Axum node communicates via local IPC loops. Incoming payloads are instantly validated, sanitized, and pushed securely into a local SQLite staging layer, surfacing inside the application’s “Inbox Tab.”

4.2 Dynamic Socket Allocation and Collision Mitigation

Binding services to rigid static addresses (e.g., `localhost:8080`) creates port collision faults if another background process occupies that socket. RoleTect resolves this liability by implementing an intelligent, sequential socket negotiation protocol across an active 20-port scanning envelope:

$$\text{Port_Pool} = \{P_{\text{start}}, P_{\text{start}} + 1, P_{\text{start}} + 2, \dots, P_{\text{start}} + 19\} \quad (1)$$

On initialization, the Tokio runtime probes each sequential port configuration. The first unassigned socket flagged by the OS is instantly captured and bound to the Axum engine. This dynamic address is then broadcasted to the local user profile, removing manual port debugging or administrative configuration.

4.3 Interactive DOM Ingestion: Visual Mouse-Selector Mode

The RoleTect Helper Extension streamlines data collection via a hardware-linked visual element selector. When activating “Selector Mode,” the extension treats the live webpage DOM as an interactive map, drawing bounding boxes over content structures as the user hovers. Upon clicking a section, the extension executes a rapid script:

1. Isolates the unique DOM ID tags or parent CSS class names of the targeted area.
2. Recursively crawls underlying child nodes to extract raw textual data blocks.
3. Strips out dynamic script components, ad tracking metrics, and volatile inline styles.
4. Checks the active port generated by the desktop core and fires an HTTP POST payload directly to `localhost:[Dynamic_Port]`.

4.4 Ecosystem Adaptability: Local vs. Cloud Targets (In future)

The connection manager is engineered to handle variable routing matrices. Beyond native local discovery over the dynamic localhost pipeline, the configuration portal accommodates custom remote endpoints. Advanced users opting to deploy their Axum framework core on a private Virtual Private Server (VPS) can replace the default localhost loop with their secure cloud domain, bridging local-first sovereignty with scalable cloud configurations.

5 Processing Engines & Cross-Platform Architecture

RoleText decouples document generation from corporate SaaS networks by integrating a local-first orchestration layer alongside an embedded cross-platform compilation engine. This architecture guarantees document rendering viability across multiple os.

5.1 The Multi-Provider AI Architecture via Rig

Rather than binding to a single AI vendor, RoleText deploys the Rust-native **Rig Framework** as a type-safe, model-agnostic abstraction layer. Candidates connect their preferred endpoint through the security panel, controlling resource allocation on a strict pay-per-token basis:

- **Commercial & Enterprise Cloud:** Seamless connectivity to OpenAI, Anthropic, Gemini, and encrypted AWS Bedrock nodes.
- **Custom & Regional Engines:** Base URL parameter mapping for specialized deep-reasoning layers like DeepSeek or Kimi.
- **Air-Gapped Privacy Nodes:** Full integration with local Ollama daemons to execute open-weights models (e.g., Llama 3) entirely on-device with zero network footprint and zero token fees.

The Rig framework handles dual-context generation loops. The raw text payload (`raw_job_content`) is analyzed alongside structural system prompts. Rig requests a structured JSON payload containing tracking metadata attributes (`company_name`, `job_title`, `target_salary`) while maintaining the core resume text within volatile operational memory.

5.2 Local Tectonic Typesetting and Self-Healing Error Loops

On desktop systems, RoleText links natively to the **Tectonic** engine—an advanced, cached typesetting ecosystem derived from **XeTeX**. Following an initial online bootstrapping phase that downloads and stores standard macros and style templates, all document processing executes completely offline within sub-millisecond compile loops.

If the LLM generates syntactically broken $\text{\LaTeX}$ markup, RoleText mitigates system faults through a state-driven self-healing recovery routine:

$$\text{State}_{\text{error}} \longrightarrow \text{Rig}(\text{Code}_{\text{broken}} + \text{Log}_{\text{Tectonic}}) \longrightarrow \text{Code}_{\text{healed}} \longrightarrow \text{Compile}_{\text{retry}} \quad (2)$$

The application intercepts the console compilation error output, dynamically logging it to the `resume_errors` or `coverletter_errors` fields within the SQLite schema. Instead of halting, the interface passes the **broken code and the precise compilation log** back to the **Rig framework**. The model outputs a structural delta patch, executes a background re-compilation, and—upon a verified successful render—wipes the error database entry clean.

5.3 Cross-Platform Mobile Strategy: The Tauri v2 Typst (Future implementation)

To scale the platform across mobile target infrastructures, RoleText will leverages the unified runtime properties of the **Tauri v2** core ecosystem, enabling successful deployment setups like verified Android APK binaries.

Because executing **multi-gigabyte XeTeX** compilation layers via Tectonic introduces significant hardware boundaries on **ARM-based mobile** devices, RoleTect employs a highly efficient **dual-engine** rendering strategy. On mobile form factors, the document compilation workflow transitions seamlessly onto a native **Typst** processing core. Typst's streamlined syntax requires far lower system footprints than traditional **L^AT_EX**, providing an ultra-fast, linear code environment that enables the Rig framework to generate error-free professional documents on mobile devices without thermal or processing performance penalties.

6 Hybrid Cloud Strategy (Ongoing) & Performance Analysis

While a local-first architecture effectively secures a candidate's data sovereignty during the document drafting phase, tracking real-world application statuses presents a technical limitation. **Local systems cannot detect external recruiter interactions once a file leaves the host machine.** RoleTect resolves this boundary through an elegant hybrid cloud expansion pathway.

6.1 The Next.js + Axum Cloud Analytics Vector

To achieve deep document lifecycle tracking without sacrificing user privacy, RoleTect maps its internal database schema onto a custom-deployable cloud instance. Users can host an **identical web-compiled Axum backend** paired with a high-performance Next.js interface on a **private Virtual Private Server (VPS)** for heavy users.

This configuration **enables secure, isolated event-tracking hooks.** When a resume or cover letter is deployed via the VPS cloud node, the system **injects transparent, unique parameters into outbound communication strings.** This allows the user to log when a PDF document link is opened or verify email delivery statuses in a **private, dedicated dashboard**—entirely eliminating reliance on commercial marketing software that harvests applicant behaviors.

6.2 Data Management: Backup, Restore, Synchronization & Staging

To maintain seamless data consistency across local desktop environments, mobile devices, and private VPS clusters, RoleTect implements a robust physical storage configuration for syncing SQLite databases:

- **The Overwrite Restore Routine:** Replaces the target device's active database file entirely with a clean, timestamped snapshot from the master machine.
- **The Temporal Merge Routine:** Analyzes the internal transaction logs chronologically, stitching unique job rows and document variants from both databases together without risk of data corruption.

Furthermore, incoming data from the browser extension enters a dedicated **Inbox Staging Table.** This acts as an isolated buffer, **preventing unparsed or raw data fragments from cluttering active application vaults** until the user explicitly prompts the system to execute a Rig compilation loop.